

PROJECT MIRROR

10–15 分钟 DEMO 实施指南

Godot 4.x (.NET) + C# · 背景 / 原版 Bash / Limit Bash

正式版垂直切片 · PC 1920 × 1080 · 可离线完成核心体验

正式版对齐原则

本文件以“人工设计/正式版设计.docx”为内容权威。章节、时长、交互限制、博弈规则、AI 方案、UI 风格与制作阶段均按正式版校正。

REDESIGNED FORMAL EDITION · 2026-08

目录与阅读路径

- 00** 范围、时间脚本与验收
- 01** 工程环境与目录
- 02** 游戏进入页、主场景与 UI 节点树
- 03** 原版 Bash 与 Limit Bash 规则层
- 04** DEMO 流程状态机
- 05** 局内统计与结算
- 06** 对话、数据与存档
- 07** UI 演出与音频
- 08** 测试清单与扩展路径

0. 范围、时间脚本与验收

DEMO 边界

DEMO 从游戏进入页开始，并按完整教学顺序验证约 10–15 分钟的最小可玩内容：选择新游戏或继续游戏后，先了解 Project X、S-17 与 Tutor 的背景，再完成原版 Bash 游戏 1 的两轮对局，最后进入 Limit Bash 游戏 2。Limit Bash 需累计两次玩家胜利才结束；若玩家连续两局平局，也按发现必不败策略结束。首版不依赖 LLM，不推断玩家后续选择，不实现 Chapter 2–6，也不训练 AI 2 / AI 3。

时间	玩家体验	完成标志
0:00–0:20	游戏进入页：新游戏 / 继续游戏 / 退出	按钮状态正确；新建或恢复有效存档
0:20–1:30	背景介绍：Project X、S-17、Tutor 与测试目的	玩家理解身份、场景目标和三选一交互
1:30–2:15	原版 Bash 规则教学	理解轮流拿 1–3 格与“拿最后一格者失败”
2:15–4:15	Bash 游戏 1 · 第一轮	N 从 12 / 16 / 20 随机；玩家先手；完成胜负结算
4:15–6:00	Bash 游戏 1 · 第二轮	N 从 13 / 15 / 17 随机；Tutor 先手；完成胜负结算
6:00–6:30	规则过渡	明确说明轮流行动将改为双方暗选、同时揭示
6:30–9:00	Limit Bash · 第 1 / 2 局	第 1 局引导玩家胜或平；第 2 局引导玩家负或平
9:00–13:30	Limit Bash · 第 3 局起	每局以 50% 概率选择玩家有利 / Tutor 有利目标，直至两胜或连续两平
13:30–14:30	局内总结	分别汇总 Bash 与 Limit Bash 的结果、局数、回合数与完成原因
14:30–15:00	重玩与退出	可从完整流程重玩；退出稳定且统计清零

0.1 P0 验收问题

1. 游戏进入页是否在启动后立即出现，并正确显示新游戏、继续游戏和退出？
2. 继续游戏是否只在存在未完成且校验通过的存档时出现，并能恢复到稳定检查点？
3. 玩家是否能先完成原版 Bash 的轮流拿取，再理解 Limit Bash 的暗选、确认与同时揭示？
4. Bash 游戏 1 的两轮是否分别正确使用玩家先手和 Tutor 先手？
5. 固定策略对手的选择是否只依据当前合法状态与可复现随机种子？
6. 发生 $A = B$ 时是否严格按正式版回溯规则处理？
7. Limit Bash 是否严格执行第 1 局胜或平、第 2 局负或平、第 3 局起按局 50% 分配目标？
8. Limit Bash 是否仅在累计两次玩家胜利或连续两次平局时结束？
9. 结束页是否能准确汇总胜 / 负 / 平局、局数与回合数？

1. 工程环境与目录

项	建议
引擎	Godot 4.x .NET 稳定版
语言	C#；目标框架跟随所选 Godot .NET 版本
窗口	默认 1920 × 1080；Canvas Items 使用 stretch，UI 采用容器布局
输入	UIUp / UIDown / UIAccept / UIBack；ChoiceLeft / ChoiceCenter / ChoiceRight / Confirm / DialogueAdvance

项	建议
提交	工程必须能在 Windows 上从干净克隆直接编译与运行

```

res://
├─ scenes/
│   ├─ Main.tscn
│   ├─ ui/TitleScreen.tscn
│   └─ ui/GameplayHUD.tscn
├─ scripts/
│   ├─ core/DemoFlowController.cs
│   ├─ games/BashGame.cs
│   ├─ games/LimitBashGame.cs
│   ├─ ai/StrategyEngine.cs
│   ├─ ai/OutcomeDirector.cs
│   ├─ session/SessionStats.cs
│   ├─ session/DemoSaveState.cs
│   ├─ data/DialogueRepository.cs
│   ├─ data/SaveGameService.cs
│   ├─ ui/TitleScreen.cs
│   └─ ui/GameplayHUD.cs
├─ data/
│   ├─ dialogue/intro.json
│   ├─ dialogue/tutorial.json
│   ├─ rules/bash.json
│   └─ rules/limit_bash.json
└─ assets/
    ├─ portraits/
    ├─ backgrounds/
    ├─ ui/
    └─ audio/

```

工程卫生

规则、表现、数据和对话必须分层。UI 不直接计算胜负；对话 JSON 不包含可执行表达式；调试数据不得写入正式存档。

2. 游戏进入页、主场景与 UI 节点树

```
Main (Control)
├─ TitleScreen
│   ├── BrightBlurredBackground
│   ├── TitleBrand
│   └─ MenuGlass
│       ├── NewGameButton
│       ├── ContinueButton
│       ├── SettingsButton (Visible = false)
│       └─ QuitButton
└─ GameRoot (hidden at startup)
    ├── BackgroundLayer
    ├── CharacterLayer
    │   └─ TutorPortrait
    ├── HUD (GameplayHUD)
    │   ├── PhaseBanner
    │   ├── RemainingCounter
    │   ├── TurnBanner
    │   ├── ResultSummary
    │   ├── DialoguePanel
    │   ├── ChoiceRow
    │   │   ├── ChoiceLeft
    │   │   ├── ChoiceCenter
    │   │   └─ ChoiceRight
    │   ├── ConfirmButton
    │   ├── TimerPanel
    │   └─ RoundResultOverlay
    └─ OverlayLayer
        ├── ChapterSummary
        └─ RuleTransition
```

2.1 游戏进入页设计

应用启动后先显示 TitleScreen，不直接进入背景对话。页面沿用透明玻璃视觉，但将参考图中的深色底改为高亮、冷白与浅青为主的失焦场景，使按钮、标题和存档状态像悬浮在玻璃层上。

区域	1920 × 1080 排布	视觉与交互要求
背景	全屏铺满；中心焦点略偏右	亮色场景模糊 12–18 px，并保留轻微光流；不使用纯黑大底
游戏标题	右上安全区，距右 8%、距上 10%	PROJECT MIRROR；大号深青灰，配一条细冷青光带
菜单玻璃	左侧 12%–42%，垂直居中	浅色半透明玻璃、24 px 圆角、柔和阴影；按钮宽度一致
存档提示	菜单下方一行	有存档时显示阶段和更新时间；无存档时显示“尚无进行中的游戏”

按钮	显示规则	点击行为
New Game	始终显示	无存档时直接创建；有未完成存档时先确认覆盖，再创建新会话存档
Continue	仅当未完成存档存在且校验通过时显示	从最后稳定检查点恢复；无存档、已完成或损坏且无法恢复时隐藏
Settings	DEMO 中隐藏，保留节点与正式版路由	DEMO 不开放音量、分辨率和语言面板
Quit	始终显示	从进入页直接退出；游戏内返回进入页前先完成安全存档

默认焦点

存在可继续存档时默认焦点落在 Continue；否则落在 New Game。键盘、手柄和鼠标必须共用同一焦点顺序，UIBack 不得误触发新游戏。

- Idle：按钮保持 70%–80% 玻璃亮度，边框为低亮冷青；文字和命中区不移动。

- Hover / Focus: 160–220 ms ease-out, 亮度提高约 8%, 边框增强并上浮 2 px, 最大缩放 1.02; 播放轻提示音。
- Pressed: 80–120 ms 缩放至 0.98, 再回弹至 1.00; 动画结束后才执行路由, 避免双击重复创建存档。
- Disabled / Hidden: Continue 不可恢复时直接隐藏而非灰显; 恢复校验期间显示短暂加载态并锁定全部按钮。
- 减少动态效果开启时取消缩放和上浮, 只保留边框、亮度与清晰焦点环。

2.2 游戏内布局规则

- ChoiceRow 始终保持三等分; 左 / 中 / 右不能因文本长度改变按钮位置。
- ConfirmButton 独立位于三选项下方, 未选择时不可用。
- PhaseBanner 必须明确显示 “背景 / Bash 第一轮 / Bash 第二轮 / Limit Bash”, 避免玩家混淆两套规则。
- TurnBanner 在 Bash 阶段显示 “玩家回合 / Tutor 回合”; Limit Bash 阶段改为 “选择 / 已锁定 / 揭示”。
- ResultSummary 分别显示 Bash 与 Limit Bash 的胜 / 负 / 平局和回合数, 不得遮挡 Tutor 表情。
- DialoguePanel 每行控制在舒适阅读长度; 字幕加速与跳过不占用三选一位置。
- 使用容器、锚点与最小尺寸适配 16:9, 不用硬编码坐标复制整套界面。

2.3 游戏内 UI 状态

状态	绿色	黄色	红色	确认
Idle	弱发光	弱发光	弱发光	禁用
Selected	所选项提高亮度	所选项提高亮度	所选项提高亮度	启用
Locked	保持选择	保持选择	保持选择	显示等待
Reveal	显示双方结果	显示双方结果	显示双方结果	隐藏
Result	冻结输入	冻结输入	冻结输入	显示继续

3. 原版 Bash 与 Limit Bash 规则层

教学顺序不可跳过

玩家必须先通过原版 Bash 游戏 1 理解“拿取数量、轮流行动、拿最后一格者失败”，然后才进入 Limit Bash，学习“双方暗选、同时揭示、不能重复上一轮与平局回溯”。

3.1 原版 Bash 游戏 1

轮次	初始格数 N	先手	通过条件
第一轮	从 12 / 16 / 20 随机	玩家	玩家获胜后进入第二轮；失败则重开第一轮
第二轮	从 13 / 15 / 17 随机	Tutor	玩家获胜后播放规则过渡并进入 Limit Bash

- 双方轮流行动，每次只能拿取 1、2 或 3 格，且不能超过当前剩余格数。
- 拿走最后一格的一方立即失败；结算后必须锁定输入，避免重复行动或负数剩余量。
- 玩家回合只启用合法按钮；例如 Remaining = 2 时，拿 3 必须禁用并显示原因。

- Tutor 使用 StrategyEngine 选择合法动作；固定随机种子必须能复现完整对局。

```
public enum Actor { Player, Tutor }
public enum GameKind { Bash, LimitBash }
public enum RoundOutcome { Continue, PlayerWin, PlayerLose, Draw }

public sealed class BashGame
{
    public int Remaining { get; private set; }
    public Actor CurrentTurn { get; private set; }

    public bool CanTake(int take) =>
        take is >= 1 and <= 3 && take <= Remaining;

    public RoundOutcome ApplyTake(Actor actor, int take)
    {
        if (actor != CurrentTurn || !CanTake(take))
            throw new InvalidOperationException();

        Remaining -= take;
        if (Remaining == 0)
            return actor == Actor.Player
                ? RoundOutcome.PlayerLose
                : RoundOutcome.PlayerWin;

        CurrentTurn = actor == Actor.Player ? Actor.Tutor : Actor.Player;
        return RoundOutcome.Continue;
    }
}
```

3.2 原版 Bash 完整回合流程

1. 按轮次配置随机选择 N，并设置先手方；HUD 同时更新阶段、剩余格数与当前回合。
2. 玩家回合启用合法的 1 / 2 / 3 按钮；确认后立即锁定输入并调用

BashGame.ApplyTake。

3. Tutor 回合由 StrategyEngine 选择合法动作；先显示思考状态，再播放拿取动画。
4. 若 $\text{Remaining} > 0$ ，则把回合权交给另一方；若 $\text{Remaining} = 0$ ，则当前行动者失败。
5. 第一轮胜利后进入第二轮；第二轮胜利后展示两套规则差异，再进入 Limit Bash。
6. 玩家在任一轮失败时只重开当前轮，不重复播放完整背景介绍。

3.3 Limit Bash

初始 N 在 20–30 之间随机；双方每轮暗选 1–3，且不能与自己上一轮相同。双方都确认后才同时公开选择，分别记为 A 与 B 。

```
if (player.Take == playerPrevious) reject;
if (opponent.Take == opponentPrevious) reject;

if (player.Take + opponent.Take < remaining)
    remaining -= player.Take + opponent.Take;
else if (player.Take != opponent.Take)
    largerTakeLoses();
else
    compareEarlierRoundsUntilDifferentOrDraw();
```

- 若 $A + B < N$ ，则扣除 $A + B$ 并进入下一轮。
- 若 $A + B \geq N$ 且 $A \neq B$ ，则数字较大的一方失败。
- 若 $A = B$ ，则逐轮向前回溯；第一次出现差值时，数字较大的一方失败。
- 若从开局起每轮都相同，则本局记为平局并重新开始。

- 规则层不读取 UI 节点；所有随机值可注入种子；每轮记录双方选择与 remaining 供结算审计。

3.4 游戏 2 的局序与终止条件

术语约定

本节把一次从初始 N 到胜 / 负 / 平结算称为“局”，把双方在该局内的一次暗选与揭示称为“回合”。胜负控制目标按局固定，不能在同一局中途更换。

Limit Bash 局序	后台结果目标	结算后处理
第 1 局	优先玩家胜；无法合法保证时允许平局	记录结果；不论胜或平都进入第 2 局
第 2 局	优先玩家负；无法合法保证时允许平局	累计胜局或平局连击；检查终止条件
第 3 局及以后	每局开始时以 50% 概率选择玩家有利或 Tutor 有利目标	继续开局，直至累计两胜或连续两平

- 正常完成：LimitBashPlayerWins $\geq$ 2；胜局可以不连续。
- 策略完成：ConsecutiveLimitBashDraws $\geq$ 2；表示玩家连续两局保持必不败。
- 任何胜局或负局都会把连续平局计数清零；单次平局不等于完成。
- 第 3 局起的 50% 是“每局结果目标”的公平抽样，不是每个回合重新掷骰，也不是承诺最终实测胜率精确为 50%。
- 达到任一终止条件后才进入 Summary；否则 RoundResult 播放完成后返回 LimitBash 并创建新局。

3.5 胜负控制的限制

限制	必须遵守的实现边界	禁止做法
读取时机	玩家按下确认且 <code>PlayerChoiceLocked = true</code> 后, OutcomeDirector 才能读取本回合选择	读取 hover、未确认选项、输入设备或局外资料
动作合法性	仅从 <code>LimitBashGame.GetLegalTutorActions()</code> 返回的动作中选择	重复 Tutor 上回合选择、越界拿取或绕过规则验证
状态完整性	锁定后只提交 Tutor 的合法动作; N、历史、玩家选择均保持原值	改写剩余数量、历史或玩家已经确认的选项
结果目标	目标在每局开始时确定并写入审计记录; 局内保持不变	发现局势不利后临时换目标, 或每回合重抽 50%
可达性	开局先验证允许结果集合可达; 首选目标不可保证时转为平局, 但不得越出本局允许集合	为继续流程而接受第 1 局玩家负或第 2 局玩家胜
数据边界	只使用当前局公开规则、锁定选择和该局历史; 需要续玩时随活动存档保存	根据局外行为推断后续选择、建立玩家画像或读取现实身份
故障处理	无可行合法动作、状态不一致或搜索超限时中止并无计数重开当前局; 重试次数有上限	静默跳过、伪造结算、无限重试或污染统计

透明度要求

界面仍按“双方暗选、同时揭示”演出; 后台在玩家锁定后读取选择并计算 Tutor 的合法动作, 是本 DEMO 明确规定的脚本化控制机制。该机制不能访问尚未确认或本局之外的数据。

3.6 对应程序实现

```
public enum OutcomeDirective
{
    PlayerWinOrDraw,
    PlayerLoseOrDraw
}

public sealed record Game2Progress(
    int GamesPlayed,
    int PlayerWins,
    int ConsecutiveDraws);

public static OutcomeDirective GetDirective(
    Game2Progress progress,
    RandomNumberGenerator rng)
{
    if (progress.GamesPlayed == 0)
        return OutcomeDirective.PlayerWinOrDraw;

    if (progress.GamesPlayed == 1)
        return OutcomeDirective.PlayerLoseOrDraw;

    // 第 3 局起：每局只抽样一次，固定至该局结束。
    return rng.Randf() < 0.5f
        ? OutcomeDirective.PlayerWinOrDraw
        : OutcomeDirective.PlayerLoseOrDraw;
}

public static bool IsGame2Complete(Game2Progress progress)
{
    return progress.PlayerWins >= 2
        || progress.ConsecutiveDraws >= 2;
}
```

OutcomeDirector 在玩家确认与公开动画之间运行。它枚举 Tutor 当前所有合法动作，复制状态并模拟每个候选，再用带缓存的有限状态求解器评估后续可达结果；规则状态本体只在最终候选选定后推进一次。

```
public int ChooseAfterPlayerLock(
    LimitBashState state,
    int lockedPlayerTake,
    OutcomeDirective directive)
{
    if (!state.PlayerChoiceLocked)
        throw new InvalidOperationException("Player choice is not locked.");
    var candidates = game.GetLegalTutorActions(state)
        .Select(take => new Candidate(
            take,
            game.Simulate(state, lockedPlayerTake, take),
            solver.SolveContinuation(state, lockedPlayerTake, take)))
        .Where(item => CanReachAllowedOutcome(item, directive))
        .ToArray();

    if (candidates.Length == 0)
        throw new ControlledOutcomeUnavailableException();
    return candidates
        .OrderByDescending(item => Score(item, directive))
        .ThenBy(item => item.Take) // 固定种子下保持可复现。
        .First()
        .Take;
}
```

指令	候选结果优先级	说明
PlayerWinOrDraw	可保证玩家胜 > 可保证平局 > 允许集合内的未决延续	玩家负候选先过滤；用于第 1 局及玩家有利抽样
PlayerLoseOrDraw	可保证玩家负 > 可保证平局 > 允许集合内的未决延续	玩家胜候选先过滤；用于第 2 局及 Tutor 有利抽样

求解器以 Remaining、双方上次选择和差异历史摘要为缓存键，执行有界动态规划 / minimax；评分只排序合法候选，最终胜负仍由 LimitBashGame.Resolve() 按正式规则计算。每局记录 GameIndex、指令、Seed、锁定选择、合法候选和真实结算，用于复现与测试。

4. DEMO 流程状态机

```
public enum DemoPhase
{
    TitleScreen,
    Background,
    BashTutorial,
    BashGame1Round1,
    BashGame1Round2,
    RuleTransition,
    LimitBash,
    RoundResult,
    Summary,
    Complete
}
```

阶段	进入条件	退出条件
TitleScreen	应用启动或完整流程结束	新游戏进入 Background；继续游戏进入存档 ResumePhase
Background	新游戏存档创建完成	Project X、S-17 与 Tutor 背景介绍完成
BashTutorial	背景介绍完成	玩家理解轮流拿取与拿最后者失败
BashGame1Round1	教学完成	玩家完成并赢得玩家先手轮次
BashGame1Round2	第一轮通过	玩家完成并赢得 Tutor 先手轮次
RuleTransition	Bash 游戏 1 通过	已说明暗选、同时揭示、禁止重复与回溯
LimitBash	规则过渡完成或上一局未达终止条件	累计两次玩家胜利，或连续两局平局

阶段	进入条件	退出条件
RoundResult	单局结算完成	播放结果后检查进度；未完成则创建下一局
Summary	达到两胜或连续两平	统计项与完成原因核对完成
Complete	确认结束	存档标记完成后返回 TitleScreen；可新游戏或退出

流程控制

DemoFlowController 是唯一能切换阶段的对象。TitleScreen 只发出新游戏、继续游戏和退出请求，SaveGameService 只读写快照；二者都不能自行跳转游戏阶段。控制器保存 Bash 轮次、Limit Bash 局数、玩家胜局和连续平局；BashGame 与 LimitBashGame 只返回规则结算，OutcomeDirector 只选择合法 Tutor 动作。

```

private void FinishLimitBashGame(RoundOutcome outcome, int rounds)
{
    Stats.Record(GameKind.LimitBash, outcome, rounds);
    ChangePhase(DemoPhase.RoundResult);
}

private void AfterLimitBashResultAnimation()
{
    if (Stats.IsLimitBashComplete)
    {
        ChangePhase(DemoPhase.Summary);
        return;
    }

    // 结果演出结束后创建下一局，并只在此处确定整局目标。
    var directive = OutcomeDirector.GetDirective(
        new Game2Progress(
            Stats.LimitBashGamesCompleted,
            Stats.LimitBashPlayerWins,
            Stats.ConsecutiveLimitBashDraws),
        SessionRng);

    OutcomeDirector.BeginGame(directive);
    ChangePhase(DemoPhase.LimitBash);
}

```

5. 局内统计与结算

字段	用途	生命周期
Wins / Losses / Draws	顶部 HUD 与结束总结	活动存档会话
BashRoundsCompleted	控制 Bash 第一轮、第二轮与规则过渡	活动存档会话
LimitBashGamesCompleted	选择第 1 / 2 / 3+ 局的结果目标	活动存档会话
LimitBashPlayerWins	累计到 2 时完成游戏 2	活动存档会话
ConsecutiveLimitBashDraws	连续到 2 时按必不败策略完成	活动存档会话

字段	用途	生命周期
OutcomeDirective / Seed	复现每局后台目标和合法动作选择	当前对局快照
RoundsPlayed	显示对局长度	活动存档会话
RemainingHistory	复现每轮剩余数量	当前对局快照
ChoicePairs	审计双方合法选择与回溯	当前对局快照
LastOutcome	驱动结算动画和固定台词	当前对局快照
External identity	现实身份、文件或设备信息	不读取

5.1 统计对象

SessionStats 只保存完成演示所需的规则事实和结算计数。运行时对象保留在内存，安全检查点会写

入纯数据快照；继续游戏时从快照重建，不序列化 Godot 节点。新游戏会创建新的 SessionStats

和 Saveld，固定种子只供开发测试使用。

```

public sealed class SessionStats
{
    public int Wins { get; private set; }
    public int Losses { get; private set; }
    public int Draws { get; private set; }
    public int BashRoundsCompleted { get; private set; }
    public int LimitBashGamesCompleted { get; private set; }
    public int LimitBashPlayerWins { get; private set; }
    public int ConsecutiveLimitBashDraws { get; private set; }
    public int RoundsPlayed { get; private set; }

    public bool IsLimitBashComplete =>
        LimitBashPlayerWins >= 2
        || ConsecutiveLimitBashDraws >= 2;

    public void Record(GameKind game, RoundOutcome outcome, int rounds)
    {
        RoundsPlayed += rounds;
        if (outcome == RoundOutcome.PlayerWin) Wins++;
        else if (outcome == RoundOutcome.PlayerLose) Losses++;
        else Draws++;

        if (game == GameKind.Bash && outcome == RoundOutcome.PlayerWin)
            BashRoundsCompleted++;
        if (game != GameKind.LimitBash)
            return;

        LimitBashGamesCompleted++;

        if (outcome == RoundOutcome.PlayerWin)
            LimitBashPlayerWins++;

        ConsecutiveLimitBashDraws =
            outcome == RoundOutcome.Draw
                ? ConsecutiveLimitBashDraws + 1
                : 0;
    }
}

```

数据边界

DEMO 存档只保留当前未完成会话的进度、规则事实、结果目标和结算计数，不读取摄像头、设备信息或现实身份。完成流程后将存档标记完成并隐藏 Continue；正式构建不保留 debug 记录。

6. 对话、数据与存档

数据类型	位置	原则
规则配置	data/rules/	数值可调，代码负责验证边界
固定台词	data/dialogue/*.json	按阶段与结算结果使用稳定 ID
会话统计	SessionStats + DemoSaveState	内存运行，稳定检查点写入快照
结果控制审计	OutcomeAudit	活动存档保存局序、指令、种子、合法候选与真实结算
调试记录	开发构建可选 JSON	结束后清理且不进入正式构建
正式存档	user://project_mirror/demo_save.json	一个活动存档；原子写入并保留 .bak
损坏隔离	demo_save.corrupt-<time>.json	不覆盖可恢复备份；进入页给出简短提示

6.1 存档快照结构

存档是可版本化的纯数据 DTO，不保存场景节点、信号连接、动画播放位置或文件绝对路径。

ResumePhase 必须指向一个稳定检查点；继续游戏后由 DemoFlowController 重建场景与 UI。

```

public sealed record DemoSaveState(
    int SchemaVersion,
    string SaveId,
    DemoPhase ResumePhase,
    long UpdatedUnixMs,
    int SessionSeed, int RngStep, int DialogueIndex,
    SessionStatsSnapshot Stats,
    GameSnapshot? CurrentGame,
    bool IsComplete,
    string Checksum);

public sealed record GameSnapshot(
    GameKind Game,
    int Remaining,
    Actor CurrentTurn,
    int BashRoundIndex, int LimitGameIndex, int RoundIndex,
    int? PlayerPrevious, int? TutorPrevious,
    OutcomeDirective? Directive,
    IReadOnlyList<ChoicePair> ChoicePairs);

```

字段组	恢复用途	写入限制
SchemaVersion / Checksum	拒绝不兼容或不完整文件	先迁移再校验；未知新版本不强行读取
ResumePhase / DialogueIndex	恢复到背景、教学、对局或结算后的稳定入口	不得指向过场动画中间帧
SessionSeed / RngStep	恢复可复现的 N、先手与第 3 局起 50% 抽样	保存确定性游标，不依赖进程内随机对象
Stats / CurrentGame	恢复计数、剩余数量、轮次、上次选择与回溯历史	只保存规则事实，不保存 UI 控件状态
IsComplete	控制 Continue 是否显示	完成流程后置 true；已完成存档不可继续

6.2 原子写入与恢复

```
public void WriteAtomic(DemoSaveState state)
{
    Directory.CreateDirectory(saveDirectory);

    byte[] json = SerializeWithChecksum(state);
    using (var stream = new FileStream(tempPath, FileMode.Create))
    {
        stream.Write(json);
        stream.Flush(flushToDisk: true);
    }

    if (File.Exists(primaryPath))
        File.Replace(tempPath, primaryPath, backupPath);
    else
        File.Move(tempPath, primaryPath);
}
```

1. 启动：SaveGameService 先验证主存档的版本、校验值和必要字段；失败时尝试
 .bak，再失败则隔离损坏文件并隐藏 Continue。
2. 新游戏：若存在未完成存档，先显示覆盖确认；确认后创建新的 SaveId、
 SessionStats 和初始 Background 检查点，再进入游戏。
3. 检查点：完成对话页、阶段切换或一整回合规则结算后，在下一次允许玩家输入前写
 入；不在按钮已锁定、Tutor 尚未结算或动画播放中写入。
4. 继续游戏：加载最新稳定快照，先重建规则对象和统计，再重建 HUD；恢复完成前保
 持输入锁定，避免重复结算或重复播放奖励。

5. 完成流程：先写入 `IsComplete = true`，再返回 `TitleScreen`；`Continue` 隐藏，

`New Game` 可建立全新活动存档。

6. 异常退出：依赖最近一次稳定检查点恢复，不尝试保存半完成内存状态；临时文件和

备份不得位于 `OneDrive` 工程目录。

存档边界

正式构建只保留一个未完成 DEMO 会话和一个恢复备份。存档不包含 `hover` 轨迹、设备标识、现实身份或局外文件内容；删除存档后 `Continue` 立即消失。

6.3 固定对话示例

```
{
  "id": "result_player_win",
  "speaker": "TUTOR",
  "phase": "RoundResult",
  "game": "BashGame1Round1",
  "outcome": "PlayerWin",
  "text": "第一轮完成。下一轮将由我先手。"
}
```

7. UI 演出与音频

7.1 视觉令牌

角色	建议色	用途
系统 / Tutor	激光绿 + 冷青	主边框、确认、稳定信息
中风险	低饱和黄	中间选项、等待与不确定性

角色	建议色	用途
高风险	珊瑚红	右侧选项与规则警告
背景	高亮白 / 冷青	亮色失焦场景，突出透明玻璃 HUD
正文	深青灰	保证玻璃面板上的对比度

- 游戏进入页必须使用亮色失焦背景与浅色玻璃菜单；标题、按钮和存档提示在 16:9 安全区内保持稳定。
- 进入页 Hover / Focus 使用同一视觉语言；Pressed 动画结束后再执行新游戏、继续游戏或退出，防止重复触发。
- UI 光效持续缓慢流动，但按钮命中区和文字不得漂移。
- Hover 只改变边框与亮度；确认前允许改选并记录次数。
- 背景图片采用高亮、失焦处理；关键文字位于浅色玻璃面板上并保持足够对比度。
- Bash 阶段确认后立即锁定输入并交替显示玩家 / Tutor 回合；Limit Bash 才使用双方锁定后同时揭示。
- 每次结算都先冻结输入，再更新剩余数量、规则类型和胜 / 负 / 平局计数。
- 主体 BGM 保持轻柔耐听，结算只使用短促且可区分的提示音。

8. 测试清单与扩展路径

类别	必测项
进入页	启动默认显示、Continue 可见性、New Game 覆盖确认、Settings 隐藏、Quit

类别	必测项
按钮动效	鼠标 Hover、键盘 / 手柄 Focus、Pressed 回弹、快速双击、减少动态效果
输入	键鼠 / 控制器映射、焦点顺序、重复确认、无选择确认、字幕加速
原版 Bash	N 候选集、玩家 / Tutor 先手、非法拿取、拿最后者失败、回合锁定
Limit Bash 规则	N = 20 / 30 边界、A + B = N、连续相同选择、全局平局回溯
局序控制	第 1 局只出现玩家胜 / 平；第 2 局只出现玩家负 / 平；目标整局不变
50% 抽样	第 3 局起按局抽样；固定种子可复现；大样本两类目标各占 45%~55%
终止条件	累计两次玩家胜利结束；连续两次平局结束；非平局会清空连续平局计数
后台动作	只在玩家锁定后读取；所有 Tutor 动作合法；不改写 N、历史或玩家选择
对手容错	无合法动作、搜索超限、边界状态、固定种子和策略回退均不会伪造结算
UI	16:9 缩放、文本溢出、色弱可辨识、按钮焦点、动画中断
存档	首次启动、原子替换、.bak 恢复、损坏隔离、版本不兼容、稳定检查点和完成后隐藏 Continue
数据	JSON 缺字段、重复 ID、非法条件、校验失败、磁盘写入失败和无权限目录
流程	TitleScreen→背景→Bash 两轮→规则过渡→Limit Bash 多局→总结→TitleScreen 的强制顺序

8.1 推荐制作顺序

1. P0: BashGame、LimitBashGame、OutcomeDirector 与自动化测试。
2. P1: TitleScreen、SaveGameService 与新游戏 / 继续游戏灰盒流程。
3. P2: 三选一 + 确认的灰盒 UI。
4. P3: DemoFlowController、背景、Bash 两轮、规则过渡和 Limit Bash 多局流程。
5. P4: SessionStats、存档检查点、两胜 / 连续两平判定与局内结算。

- 6. P5: 固定对话 JSON 与结果台词。
- 7. P6: 亮色失焦首页、Tutor 立绘、透明 HUD、光效与音频。
- 8. P7: 完整 10–15 分钟走查、退出恢复与可用性测试。

8.2 DEMO 通过后

DEMO 通过后，先扩充章节内容、场景资产与完整设置，再按正式版技术方案接入进阶 AI。当前演示包始终保持离线、脚本化结果目标与局内统计边界，后续能力必须在独立分支和单独验收中实现。

阶段	新增范围	进入下一阶段前必须证明
DEMO	游戏进入页 + 可恢复存档 + 背景 + 原版 Bash 两轮 + Limit Bash 多局	Continue 恢复准确；完整顺序不可跳过；两胜 / 连续两平终止正确
试玩版 1	扩充 Chapter 0 / 1 与美术、设置	完整流程稳定，资源与本地化可审计
试玩版 2	增加进阶对手与更多章节	新能力与 DEMO 边界隔离，回归测试通过
正式版	完整 Chapter 0–6 + Ending	模型、内容、联网、隐私、多语言与可访问性全量验收

最终实施判断

当玩家能从游戏进入页新建或继续有效存档，无需说明书即可理解背景，完成原版 Bash 游戏 1 的玩家先手与 Tutor 先手两轮，再在 Limit Bash 游戏 2 中累计两次胜利或连续两次平局，完成总结并返回进入页时，垂直切片才算通过。